

Serialization & Deserialization — Lecture Notes

1. The Core Problem

A JavaScript client and a Rust server need to exchange data over HTTP. The issue: they have **completely different type systems**.

- JavaScript is dynamic and interpreted — no strict types.
- Rust is statically typed and compiled — strict, low-level types.

How does data sent from one end get understood at the other end, across different languages, machines, and network hops?

2. The Solution: A Common Standard

The answer is to agree on a **shared serialization standard** — a set of rules both sides follow when sending and receiving data.

```
JavaScript Object → [serialize] → Common Format → [transmit] →  
Rust Struct  
Rust Struct      → [serialize] → Common Format → [transmit] →  
JavaScript Object
```

- **Serialization:** Converting from a language-native data type → common format (for transmission or storage).
- **Deserialization:** Converting from common format → language-native data type (after receiving).

Key property: The common format must be **language-agnostic** — any language/platform can read and write it.

3. Where This Fits in the OSI Model

As a backend engineer, you work at **Layer 7 (Application Layer)**. Below that, the network stack handles conversion into IP packets, data frames, and physical bits — that is not your concern.

```
Client (App Layer) → serialize to JSON → ... network layers ... →  
bits over wire
```

Server (App Layer) ← deserialize from JSON ← ... network layers ... ← bits over wire

The data arrives back as the same format (JSON) at the application layer on both ends. Everything in between is the network's job.

4. Serialization Standards

Text-Based (Human-Readable)

Format	Notes
JSON	Most popular for HTTP/REST communication; human-readable
XML	Older; still used in enterprise and SOAP APIs
YAML	Common in config files (Docker, Kubernetes); not typical for HTTP transmission

Binary Format (Machine-Optimized)

Format	Notes
Protobuf	Most popular; used with gRPC; compact and fast
Avro, Thrift, etc.	Other binary options, less common

Text-based vs Binary tradeoff:

- Text (JSON): human-readable, easy to debug, slightly larger payload.
- Binary (Protobuf): not human-readable, much smaller and faster — preferred for high-performance internal services.

This course focus: JSON — used in ~80% of HTTP REST API communication.

5. JSON Deep Dive

JSON = JavaScript Object Notation

Despite the name, JSON is **not JavaScript** — it is a language-agnostic standard used everywhere: config files, log files, HTTP APIs, databases, and more.

Syntax Rules

```

{
  "id": 1,
  "title": "Some Book",
  "author": "John Doe",
  "available": true,
  "tags": ["fiction", "classic"],
  "address": {
    "country": "India",
    "pincode": 110001
  }
}

```

Rule	Detail
Wrapped in <code>{ }</code>	Top-level must be an object (or array)
Keys in double quotes	Keys are always strings: <code>"key"</code> not <code>key</code>
Value types	<code>string</code> , <code>number</code> , <code>boolean</code> , <code>null</code> , <code>array</code> , <code>nested object</code>
Nesting allowed	Objects can contain other objects or arrays recursively
No trailing commas	Strict syntax — trailing commas are invalid

What JSON is NOT

- Not a JavaScript object literal (JS objects can have unquoted keys, functions, etc.)
- Not a binary format — it's plain text
- Not language-specific — parseable in Python, Rust, Go, Java, etc.

6. JSON in the Client-Server Flow

Client (JS)		Server (Rust/Node/Python/etc.)
JS Object		Native struct / dict / class
↓ JSON.stringify() / serialize		↓ json.parse() / deserialize
JSON string	HTTP request →	JSON string
		↓ parse into native type
		Business logic + DB ops
		↓ serialize back to JSON
JSON string	← HTTP response	JSON string
↓ JSON.parse() / deserialize		
JS Object → render UI		

Demo observation from lecture:

- POST `/api/books` — request body is JSON with `"id"`, `"title"`, `"author"` (keys double-quoted, values typed).
- Server responds with a JSON array of book objects in the same format.
- Client parses the response and renders the list — the full serialize → transmit → deserialize cycle in action.

7. Summary

Term	Definition
Serialization	Converting native data → common format for transmission/storage
Deserialization	Converting common format → native data after receiving
JSON	The dominant text-based serialization standard for REST APIs
Protobuf	Dominant binary standard for gRPC / high-performance services

Serialization and deserialization exist to make data **language-agnostic** — so a JavaScript client and a Rust server (or any two systems) can communicate meaningfully without knowing anything about each other's internal type systems.

Quick Revision Checklist

- ☐ Serialization = native type → common format; Deserialization = common format → native type
- ☐ Purpose: language/domain-agnostic data exchange
- ☐ Backend engineer's concern: application layer (JSON in/out); OSI layers below are network's job
- ☐ Text-based standards: JSON, XML, YAML — JSON dominates HTTP REST
- ☐ Binary standards: Protobuf (gRPC), Avro, Thrift — faster, smaller, not human-readable
- ☐ JSON rules: `{ }` wrapper, keys must be double-quoted strings, values can be string/number/bool/null/array/object
- ☐ JSON is language-agnostic despite being named after JavaScript